A Distributed Multi-Agent AI Platform for Scalable Forensic Audit and Compliance

Podichetty Samiksha
Department of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, Telangana, India
samiksha10.1b@gmail.com

Chandana Devanaboyina
Department of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, Telangana, India
chandanad3884@gmail.com

Medapally Shiva Reddy
Department of Information Technology
Chaitanya Bharathi Institute of Technology
Hyderabad, Telangana, India

***Abstract*—Auditing procurement means checking every vendor invoice against its purchase order, goods receipt, vendor record and approval trail, and justifying each exception against a written policy. AuditBench shows that large language models (LLMs) find injected errors in financial data but explain them poorly and rarely cite the governing standard; a survey of LLM-based multi-agent systems in finance shows that systems of specialised agents suit interactive financial tasks. We present AuditIQ, a platform for the forensic audit of procurement transactions built as separate services: audit workers, a policy service, a data store, a model endpoint, and a REST API with a web application for reviewers. Its base is a multi-agent audit pipeline that follows these two works, and we add one feature: *policy-driven deterministic controls*. The system reads the organisation's policy PDF, takes every threshold from the wording of the rule that states it, and twelve controls execute these thresholds before any LLM is called; no threshold is written in code or in a prompt. Given fourteen policy documents, the system read 196 of 196 thresholds correctly and decided all 40,000 invoices of five synthetic ledgers as each policy requires, whereas the same controls with frozen thresholds decided up to 30.7% of them against the policy in force. On three benchmark ledgers the controls reached an F1 of 0.833 at a precision of 1.000, against 0.536 and 0.498 for an isolation forest and a one-class SVM, and the control stage audited 172,000 invoices per second with eight processes on one laptop. With a local 7B model, a model-assisted reader proposed 80 of 84 thresholds of six free-form policies correctly, and a 20-invoice pilot ran every LLM stage end to end. The LLM stages have not been measured at benchmark scale; for them we state expected values fixed before measurement.**

***Index Terms*—forensic audit, compliance, multi-agent systems, large language models, distributed platform, scalability, procure-to-pay, policy as data, deterministic controls, explainability**

I. Introduction

A. Background

An auditor who tests procurement spend must establish four things for every invoice: that it corresponds to an approved purchase order (PO); that the goods or services were received, as recorded in a goods receipt note (GRN); that the vendor is legitimate and tax-compliant; and that the payment was approved within the approver's delegated limit. The evidence sits in separate records, and what counts as an exception is defined in an organisation-specific policy document. Continuous auditing proposes to test every transaction automatically instead of a sample [1].

B. Problem

LLMs are a natural candidate for this work because the policy is written in natural language. AuditBench [2] shows the limits: LLMs identify errors when they are given the transaction history, but explain them poorly and rarely cite the relevant standard. Systems of several specialised LLM agents are increasingly used for financial tasks [3], and we built such a pipeline for procurement audit. In that pipeline we observed a second problem, which is the subject of this paper: *the thresholds of the policy were not under control*. The tolerance of the amount match, the daily invoice count and the approval limits were stated in policy prose, one of them was repeated as a literal number inside an agent prompt, and whether a rule check ran at all depended on an agent choosing to call a tool. When the policy changes, such a system must be edited by hand, and nothing shows that the edit agrees with the document.

C. Contribution

This paper presents AuditIQ, a multi-agent platform for forensic audit and compliance in procurement. *Forensic* means that every decision can be reconstructed afterwards: each finding carries the rule, the evidence and the document page it rests on, and every stage writes to an audit trail. *Distributed* means that the platform is a set of separate services that communicate over network interfaces (Section IV-C). We keep the base multi-agent pipeline and add one feature, *policy-driven deterministic controls*:

- the policy *document* (PDF) is the input of the system;
- every threshold is read from the wording of the rule that states it and stored in a clause registry;
- twelve deterministic controls execute the registry before any LLM is called, and contain no constant of their own;

- a document that the system cannot execute is refused with the reason.

The feature is implemented, tested and measured (Sections V–VIII), including its throughput against the number of worker processes and the size of the ledger. We also measure the controls against two classical anomaly detectors, a model-assisted reading of policies written in free prose with verification by code and confirmation by a person, and, as a pilot on 20 invoices, every LLM stage of the base system with a local model. The paper reports these measured results and, separately and clearly marked, expected values for what has not been measured.

D. Organisation

Section II reviews related work. Section III presents the two base papers. Section IV describes the base system and Section V the extra feature. Section VI gives the implementation and technology stack, Section VII the experimental setup, and Section VIII the measured and the expected results. Sections IX and X give limitations and conclusions.

II. Related Work

A. Anomaly Detection in Audit Data

Anomaly detection methods rank records by how unusual they are [4]; isolation forests [5] and one-class support vector machines [6] are common unsupervised choices, and we use both as baselines. Long et al. [7] apply machine learning to anomaly detection in the financial audit of construction projects. Such detectors return a score or a label. An unusual record is not necessarily a policy violation, and a violation, such as an invoice approved above a delegated limit, need not look unusual. A detector also learns what is normal from data, so a change of policy requires new training.

B. Large Language Models in Auditing

LLMs have been used to match passages of financial reports against audit requirements [8] and to verify the regulatory compliance of financial reports [9]. AuditBench [2] evaluates LLMs on financial statement auditing (Section III). A known weakness is that generated text can assert facts that the input does not support [10], which is not acceptable in an audit, where the evidence for every conclusion has to be shown.

C. LLM-Based Multi-Agent Systems

Systems in which several LLM agents with different roles cooperate are surveyed in general in [11] and for financial markets in [3]. Letting agents criticise one another's answers improves factual accuracy [12]. For high-stakes decisions, review of uncertain cases by a human remains the accepted safeguard [13].

D. Research Gap

The works above either detect anomalies without naming a rule, or let an LLM reason over the policy text. We found no work in which the organisation's policy document itself configures executable audit controls, so that the thresholds a system applies are provably those the document states.

III. Base Papers

This work builds on two papers. Table I summarises what each contributes and what this work adds.

A. Base Paper 1: AuditBench

What it does. Wang et al. [2] ask how well LLMs can audit financial statements. They build a benchmark that pairs real financial tables with synthesized transaction data into which errors are injected, and evaluate models in a five-stage framework; one stage asks the model to link each error to the accounting standard it violates.

What it finds. Current LLMs identify errors when given the historical transaction data, but are weak at explaining the errors and at citing the relevant accounting standards, and they struggle to complete a full audit and to make the required revisions.

What we take from it. (i) The construction of evaluation data: synthetic transactions with injected, labelled errors. (ii) The requirement that a finding must cite the rule it relies on, which AuditBench shows LLMs do poorly on their own.

What it leaves open. AuditBench measures models; it does not propose a system that removes the weakness. It concerns financial statements, not the matching of procurement documents against an organisation's policy.

B. Base Paper 2: LLM-Based Multi-Agent Systems in Finance

What it does. Liu et al. [3] survey LLM-based multi-agent systems for financial markets. They argue that traditional models struggle with the strategic and interactive nature of markets, and that multi-agent systems, characterised by heterogeneity, autonomy, adaptability and bounded rationality, fit it better. The survey covers the enabling techniques of LLM agents, the mechanisms that produce collective intelligence, applications such as market simulation, systemic risk analysis and policy evaluation, and technical and regulatory challenges.

What we take from it. The architecture of the base system: several heterogeneous agents, each with its own role and its own view of the data, whose outputs are combined.

What it leaves open. The survey concerns the simulation of markets, not audit. It names regulatory challenges of LLM agents; an audit system must answer them concretely, because every decision has to be traceable to a rule and to evidence.

IV. Base System

The base system is a multi-agent audit pipeline that follows the two base papers. Fig. 2 shows it together with the extra feature.

A. Inputs

A *transaction* x is one invoice together with the records linked to it: its PO, its GRN, the vendor master record, the approval log entries, and the vendor's other invoices. The policy is a document of numbered rules; each rule defines one exception and its severity.

TABLE I
THE BASE PAPERS AND THIS WORK

Work	Task	Contribution used here	Left open
AuditBench [2]	Benchmark of LLMs on financial statement auditing	Synthetic transactions with injected, labelled errors; findings must cite the governing rule	No system that fixes the weak citation; not procurement; no organisation policy
Liu et al. [3]	Survey of LLM-based multi-agent systems for financial markets	Architecture of heterogeneous, autonomous specialist agents	Not audit; regulatory traceability named as a challenge
This work	Transaction-level audit of invoices against an organisation's policy	Base multi-agent pipeline, plus one feature: controls executed from the policy document	LLM stages measured only in a 20-invoice pilot (Section VIII-H)

B. Stages

1) *Evidence extraction.* The invoice identifier is resolved to a transaction by joining the linked records, so that every later stage sees the same values.
2) *Specialist agents.* Three LLM agents, each with a narrow remit, examine the transaction: a *math and approvals* agent, a *tax and identity* agent and a *vendor intelligence* agent. Each returns its findings as structured output. This is the heterogeneous multi-agent design of [3].
3) *Challenger.* A further LLM agent reviews the findings and removes those the evidence does not support, in the spirit of [12].
4) *Risk score and route.* With severity weights $w = (30, 20, 10, 5)$ for critical, high, medium and low findings, the score is

$$R(x) = \min\Big(100,\ \sum_{f \in F(x)} w_{s(f)}\Big), \tag{1}$$

raised to at least 70 if $F(x)$ contains a critical finding and to at least 50 if it contains a high one. An invoice with $R(x) \geq 50$ is escalated to a human reviewer; the others are approved.
5) *Human review and audit pack.* The reviewer approves, rejects or overrides with a justification. An audit pack records the findings, their evidence, the cited rules and the decision.

C. Platform Architecture

The platform consists of five services (Fig. 1). They share nothing but the data store and communicate over network interfaces, so each can run on its own machine.

- *Audit workers* run the pipeline. One invoice is one unit of work: its transaction carries every record it needs, so auditing it requires nothing from any other audit, and workers need no coordination. Within one invoice the three specialist agents run in parallel; across invoices the batch runner audits several invoices at the same time.
- *Policy service* adopts policy documents and keeps the versions (Section V).
- *Data store* is a relational database (PostgreSQL as a server, or SQLite as a file). It holds the ledger, the policy versions and passages, and everything the audit produces.

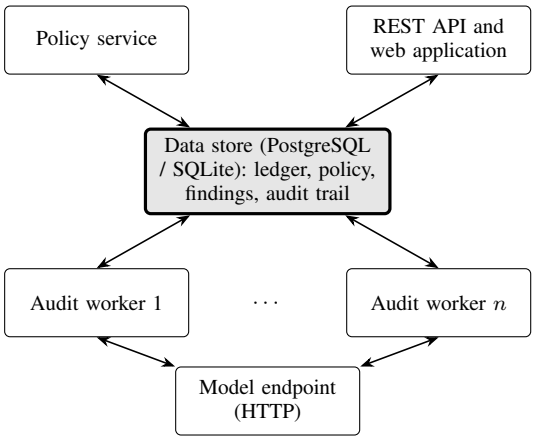


Fig. 1. Services of the platform. Audit workers are independent of one another: each takes invoices, reads the ledger and the active policy from the data store, calls the model endpoint for the judgement rules, and writes its results back.

- *Model endpoint* is any server that offers an OpenAI-compatible HTTP interface, local or hosted. Only the agents and the challenger use it.
- *REST API and web application.* An HTTP API exposes the ledger, the audit results, the review queue, the policy store and the copilot; long tasks, such as auditing the ledger again after a policy change, run as background jobs whose progress the client polls. A browser application for reviewers uses only this API (Section VI-D).

In the experiments of this paper all services ran on one laptop; a deployment on several machines has not been measured.

D. Forensic Evidence Trail

For every audited invoice the platform stores the result, every finding with its evidence, the findings that the challenger removed together with its reason, one audit-trail entry per stage, the policy version that was applied and, after review, the reviewer's decision with name and justification as the last entry of the trail. Results are added and never replaced, so an invoice that is audited again under a new policy keeps its earlier results. This is what allows a later examiner to reconstruct how a decision was reached.

TABLE II
INPUT AND OUTPUT OF EACH STEP

Step	Input	Output
Read PDF	Policy PDF	Text lines with font size, position and page
Structure	Text lines	Passages: the 16 rules and the other paragraphs, each with section and page
Extract	Passages; reference registry	Registry (16 rules, 14 values); list of changed parameters; list of problems
Adopt	Registry; PDF	New policy version in the store; passages stored for citation
Audit	Invoice identifier; active registry	Findings with rule, severity and evidence; score; route; audit trail

E. Weakness of the Base System

In the base system the agents perform every check, including those that are plain arithmetic. Three consequences follow. (i) Thresholds live in prose and prompts, so a policy change means editing the system. (ii) The text an agent reads and the number a tool applies can disagree unnoticed. (iii) Following AuditBench [2], the citation of the rule by an LLM cannot be relied on. The extra feature addresses these three points for every rule that can be decided by comparing fields.

V. EXTRA FEATURE: POLICY-DRIVEN DETERMINISTIC CONTROLS

A. Idea

Every check that can be decided by comparing fields is taken away from the LLM agents and executed as code, and that code takes all its thresholds from the organisation's policy document. The agents keep only the rules that need reading or judgement.

B. Definitions

A policy version is a tuple

$$P_v = (K,\ s,\ \kappa,\ \theta), \tag{2}$$

where K is the set of rule identifiers, $s(k)$ the severity of rule k, $\kappa(k) \in \{\text{control}, \text{judgement}\}$ its kind, and θ_k its parameters. Stored as data, the tuple is the *clause registry*. Our policy has 16 rules: 12 of kind control and 4 of kind judgement, with 14 parameter values in total.

A *control* is a function $c_k(x; \theta_k)$ of a transaction and of the parameters of its rule. It returns nothing, or a *finding*

$$f = (k,\ s(k),\ E), \tag{3}$$

where E is the set of field values on which the finding rests. A control contains no constant of its own.

C. Input and Output

Table II gives the input and the output of each step of the feature.

TABLE III
PARAMETERS, THE WORDING THEY ARE READ FROM, AND THE CONTROL THAT USES THEM

Rule	Wording in the policy (value in bold)	Control
2.1.1	difference of up to **1%** of the PO amount	amount match
3.1.1	exactly **15** alphanumeric characters	GSTIN format
3.1.2	a state code from **01** to **37**	GSTIN state code
4.1.1	at most **3** invoices dated on the same day	same-day splitting
4.1.2	as a guide, **7** days; within **15%** of the limit	(judgement rule; read by an agent)
4.2.1	dated on a **Saturday or Sunday**	non-business day
4.3.1	an exact multiple of INR **10,000**	round amount
4.4.1	registration status is **Suspended**	blocked vendor
5.1.1	table of roles and limits (**4** values)	approval limit

D. Logic

The feature works in six steps.

Step 1: read the document. The PDF is opened and every text line is taken with its font size, its left edge and its page number.

Step 2: find the rules. The segmentation uses layout, not keywords. The most frequent font size is the body size; a line at least one point larger is a heading; a line that begins with "Rule ⟨number⟩ (⟨severity⟩)," opens a rule, and indented lines continue it.

Step 3: extract the thresholds. For every parameter, one pattern is matched against the text of the rule that states it (Table III). The approval limits are read from the table of roles and amounts. The severity is read from the rule itself. The result is $\hat{\theta} = E(D)$ for document D.

Step 4: check, and refuse if necessary. The extraction is by pattern and not by an LLM, so a value is either found in the expected sentence or reported as missing. The document is refused, and nothing is stored, when (a) a rule that the system executes is absent, (b) a threshold is not found in the wording of its rule, (c) the document contains a rule for which the system has no check, (d) a value is not meaningful, for example a word that is not a day of the week, or (e) the document states no version.

Step 5: adopt. An accepted document becomes a new policy version $\hat{P} = (K, \hat{s}, \kappa, \hat{\theta})$. Its passages are stored with their page numbers. The set K and the kinds κ come from a reference version, which records what the system is able to execute.

Step 6: audit. For each invoice the controls run, and every finding cites its rule, the policy document and the page:

$$F_{\mathrm{D}}(x) = \left\{\, c_k(x; \hat{\theta}_k) \ :\ \kappa(k) = \text{control} \,\right\}. \tag{4}$$

The findings then enter the score (1) unchanged. Fig. 3 follows one threshold through these steps.

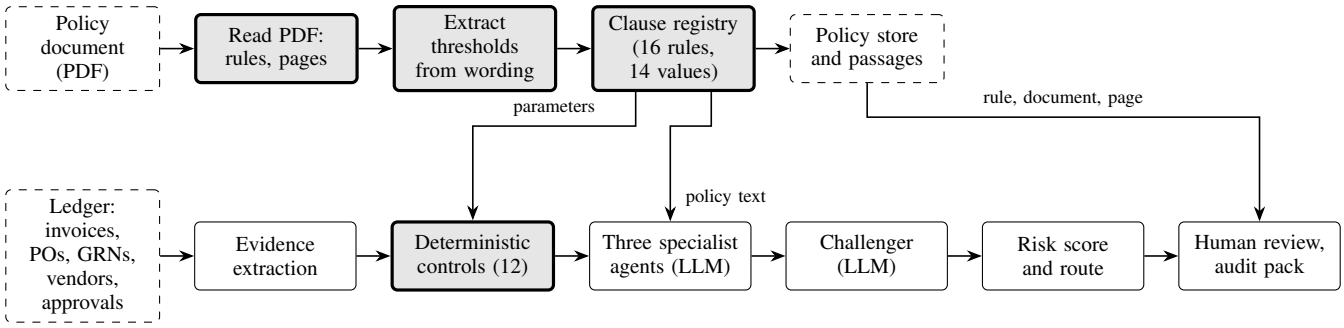


Fig. 2. The AuditIQ pipeline. White boxes are the base system; grey boxes with a thick border are the extra feature of this paper; dashed boxes are data. The policy document configures the controls, which run before any LLM is called.

Rule in the PDF (page 1). "Rule 2.1.1 (HIGH), amount match. The invoice amount must equal the PO amount. A difference of up to **0.5%** of the PO amount is tolerated."

Pattern. `difference of up to (N)% of the PO amount`

Registry. rule 2.1.1, severity high, kind control, `amount_tolerance_pct = 0.5`

Control. Invoice 100,800.00, PO 100,000.00: difference 0.80% $>$ 0.5%

Finding. Rule 2.1.1, high; evidence `invoice.amount`, `po.amount`; cites the document, page 1. Score 50: escalate.

Fig. 3. One threshold from the policy document to a finding (illustrative values). Under the original tolerance of 1% the same invoice raises no finding.

E. The Twelve Controls

The controls are: amount match (2.1.1), quantity match (2.1.2), missing PO or GRN reference (2.1.3), exact duplicate invoice number (2.2.1), GSTIN format (3.1.1), GSTIN state code (3.1.2), GSTIN against the vendor master (3.1.3), same-day splitting (4.1.1), non-business day (4.2.1), round amount (4.3.1), blocked vendor (4.4.1) and approval limit (5.1.1). Four of them need no parameter. As an example, the amount match control is

$$c_{2.1.1}(x;\tau) = \begin{cases} \text{finding}, & 100\,|a_{\text{inv}} - a_{\text{po}}| > \tau\, a_{\text{po}}, \\ \varnothing, & \text{otherwise}, \end{cases} \tag{5}$$

where a_{inv} and a_{po} are the amounts in integer paise and τ is the tolerance in percent read from the document. Integer arithmetic means that rounding cannot move a decision at the boundary. The approval limit control takes the limit of the approver's role from the registry, not from the approval log, because the log records what the approval system believed at the time and not what the policy in force says.

F. What Remains With the Agents

Four rules need reading or judgement and stay with the LLM agents of the base system: a goods-receipt remark that contradicts the billed quantity (2.1.4), an invoice description that does not match the PO (2.1.5), a near-duplicate invoice with a re-keyed number (2.2.2), and purchases structured across days to stay below an approval limit (4.1.2). The agents receive the control findings and may add findings only under these rules.

VI. Implementation

A. Technology Stack

Table IV lists the technology of each part. The feature uses no LLM.

B. Data Structures

Registry entry. Each rule is stored as its title, severity, kind and parameters, for example rule 5.1.1 (approval limit; critical; control) with the limit of each of four roles.

Passage. Each rule or paragraph read from the PDF is stored with its kind, its rule number, the heading of its section, its page and its text.

Finding. A finding holds the rule, the severity, the name of the control, a sentence that states the exception, the evidence as pairs of field and value, and the citation (document and page).

C. Operation

Adopting a policy document is one command, or one upload in the web application; auditing the ledger is a second. No code and no prompt is edited when the policy changes. Every audit stores, for each invoice, the result, the findings and one audit-trail entry per stage, together with the policy version that was applied.

D. Web Application

Reviewers work in a browser application written in React and TypeScript that uses only the REST API. Its pages are an overview of the audit; the review queue; the audit pack of one invoice, with each finding, its evidence and citation, the audit trail and the decision form; the policy, where a new document is uploaded, the changes it makes are shown before it is adopted, and a document in free prose is read by the model and confirmed by the reviewer; a copilot that answers with citations; the pipeline; and the measured results. Three views are three-dimensional, drawn with React Three Fiber [18] on

TABLE IV
TECHNOLOGY STACK

Part	Technology	Role	Status
Extra feature			
Reading the policy PDF	PyMuPDF [14]	Text lines with font size, position and page; segmentation into rules	Implemented, measured
Threshold extraction	Python regular expressions	One pattern per parameter; refusal with reasons	Implemented, measured
Clause registry	JSON; Python	Rules, severities, kinds and parameters as data	Implemented, measured
Deterministic controls	Python (no library)	Twelve functions of transaction and registry	Implemented, measured
Policy documents of the experiment	ReportLab [15]	Renders each policy variant as a PDF	Implemented
Policy and audit store	SQLite or PostgreSQL	Policy versions, passages, findings, audit trail, review decisions	Implemented
Policies in free prose (extension)	LLM; Python verification	The model proposes each value with a quote; code checks that the quote is in the document and the value in the quote; a named person confirms	Implemented, measured
Base system			
Pipeline orchestration	LangGraph [16]	Graph of stages; parallel agents; pause for the reviewer	Implemented
Specialist agents and challenger	LLM behind an OpenAI-compatible endpoint	Judgement rules; removal of unsupported findings	Implemented; 20-invoice pilot
Risk score and route	Python	Equation (1) and the threshold of 50	Implemented
REST API	FastAPI [17], Uvicorn	Ledger, results, review, policy adoption and copilot over HTTP; background jobs; optional bearer token	Implemented, tested
Web application	React, TypeScript, React Three Fiber [18], three.js [19]	Review queue, audit packs, policy adoption, results; 3D views of vendor risk, of the pipeline and of the policy change	Implemented, tested
Lightweight dashboard	Streamlit	The same review work in a simpler interface	Implemented
Evaluation			
Synthetic ledgers	Python, NumPy, pandas, Faker	Linked records with labelled anomalies, after [2]	Implemented
Independent ground truth	pandas	Recomputes the violated rules from the ledger tables	Implemented
Classical baselines	scikit-learn [20]	Isolation forest and one-class SVM with cross-fitted thresholds	Implemented, measured
Tests	pytest; Vitest, Testing Library	Python suite on SQLite and PostgreSQL; frontend suite including the 3D scenes; both in continuous integration	Implemented

three.js [19]: a *risk constellation* that places every vendor in the sector of its spending category, at a height given by the highest risk among its invoices and with a size that grows with its number of invoices, drawn as one instanced mesh so that the number of vendors does not change the number of draw calls; the pipeline as a graph whose stages light up for the chosen configuration; and the share of invoices that a frozen system decides against each policy. Every 3D view has a table with the same data beside it, and a fallback for browsers without WebGL. The API compares its bearer token in constant time, accepts only PDF uploads of up to 20 MB, and validates every request body.

E. Testing

The Python test suite (313 tests, each backend counted separately) runs on SQLite and on PostgreSQL. The 26 tests of the web application cover the API client, every page against a mocked API, and the three 3D scenes, which React Three Fiber's test renderer draws without a graphics processor. Continuous integration runs both suites, the linters, a production build and a check of the shipped packages for known vulnerabilities on every change. No test calls a model: where a stage needs one, a scripted stand-in replies, so the tests check what the pipeline asks and how it handles the reply, not how well a model audits.

VII. EXPERIMENTAL SETUP

A. Question

When an organisation changes its policy and supplies only the new document, does the system decide every invoice as the new policy requires, and how wrong is a system that is not updated?

B. Data

We use five synthetic ledgers with 40,000 invoices in total: four ledgers of 5,000 invoices and one of 20,000, each with purchase orders, goods receipts, vendors and approval logs. As in AuditBench [2], anomalies are injected into synthesized transactions and labelled when they are injected.

C. Policy Variants

Starting from the reference policy (version 2.5) we derive fourteen variants. Eleven change one threshold: the amount tolerance (1% to 0.5%, 2% and 5%), the daily invoice count per vendor (3 to 2 and 5), the Manager approval limit (100,000 to 50,000 and 150,000), the Senior Manager limit (500,000 to 250,000), the round-amount multiple (10,000 to 5,000 and 50,000), and the working week (Saturday becomes a business day). Three change several thresholds at once: a strict policy, a lenient policy, and version 2.6. Each variant is rendered as a three-page PDF, which is all the system receives.

D. Systems Compared

Policy-driven: the controls under the registry extracted from the variant's PDF. *Frozen:* the same controls with the thresholds of version 2.5; it stands for a system whose thresholds are written into prompts and code, until someone edits it.

E. Ground Truth

For each variant, the set $Y_P(x)$ of rules that an invoice violates is recomputed from the ledger tables by the validator of the data generator. It is a second implementation of the written rules and shares no code with the controls.

F. Measures

Extraction: parameter values read correctly, and rules read word for word. *Policy adherence:* the share of invoices for which the set of cited rules equals the ground truth of the policy in force,

$$A(P) = \frac{1}{N} \sum_{x} \mathbf{1}\big[\{k(f) : f \in F_{\mathrm{D}}(x)\} = Y_P(x)\big]. \quad (6)$$

For the frozen system: the findings that the policy in force does not support, the exceptions it does not raise, and the invoices it approves although the policy in force requires review (missed escalations).

VIII. Results

Subsections A to G report measured results of the feature and of the platform; Subsection H reports a measured pilot of the LLM stages on 20 invoices; Subsection I gives expected values for what has not been measured.

A. Reading the Policy Documents (Measured)

All fourteen documents were accepted. The system read 196 of 196 parameter values correctly and recovered 224 of 224 rules word for word. Reading one document took between 0.02 and 0.04 seconds.

B. Adherence to the Policy in Force (Measured)

Table V gives the result for each policy, pooled over the five ledgers. The policy-driven controls decided every invoice as the policy in force requires: no wrong decision in 560,000 invoice audits. Under the reference version the controls and the independent ground truth agree on all 40,000 invoices. A separate ledger of 5,000 invoices contains 150 invoices at the boundary between version 2.5 and version 2.6; the controls flag all 150 under version 2.5 and none under version 2.6, and no other invoice under either.

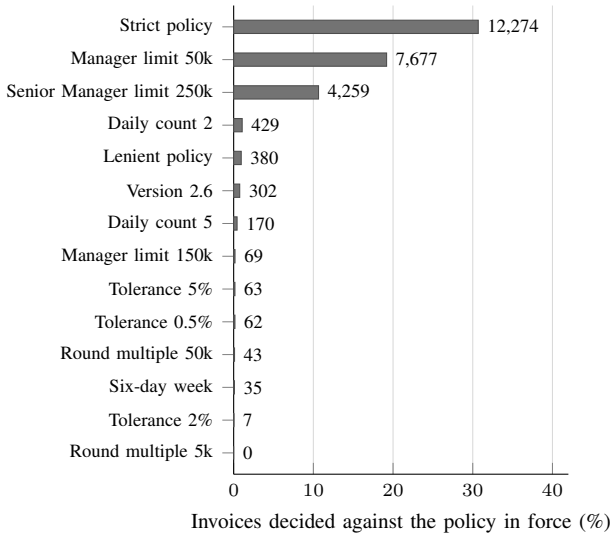


Fig. 4. Measured. Share of the 40,000 invoices that a system with thresholds frozen at version 2.5 decides against the policy in force; the number beside each bar is the count of invoices. The policy-driven controls decide none against it.

C. Cost of a Frozen System (Measured)

The number of invoices whose correct decision changes ranges from none to 12,274 of 40,000 (30.7%) under the strict policy (Fig. 4). The direction of the change decides the kind of error. A tightening leaves a frozen system silent: with the Manager limit lowered to 50,000 it does not raise 7,677 exceptions and approves 7,514 invoices that the policy in force sends to review. A loosening costs reviewer time instead: under version 2.6 a frozen system raises 302 findings that the policy no longer supports.

D. In the Application (Measured)

We repeated one change through the deployed path on the ledger of 20,000 invoices (Table VI). Adopting the strict policy document took 7 seconds, and auditing the 20,000 invoices again, with every result, finding and trail entry stored, took one minute on a laptop. No file of the system was edited between the two runs.

E. Scalability (Measured)

Ledger size. One process audited ledgers of 5,000, 20,000 and 40,000 invoices under the reference policy in 0.089, 0.389 and 0.731 seconds: 17.8, 19.4 and 18.3 microseconds per invoice. The cost per invoice does not grow with the ledger, so the time is linear in the number of invoices.

Worker processes. Because one invoice is one independent unit of work, the control stage can be split across workers without coordination. Table VII and Fig. 5 give the throughput for 300,000 invoice audits (20,000 invoices under 15 policy versions) on a laptop with 12 logical CPUs. Every configuration produced the same 16,840 findings. Throughput rose from

TABLE V
MEASURED: ADHERENCE TO THE POLICY IN FORCE ON 40,000 INVOICES PER POLICY

Policy variant	Decisions moved	Policy-driven	Frozen at version 2.5			
		Adherence	Adherence	Unsupported findings	Missed exceptions	Missed escalations
Tolerance 0.5%	62	1.000	0.998	0	62	62
Tolerance 2%	7	1.000	1.000	7	0	0
Tolerance 5%	63	1.000	0.998	63	0	0
Daily count 2	429	1.000	0.989	0	429	428
Daily count 5	170	1.000	0.996	170	0	0
Manager limit 50,000	7,677	1.000	0.808	0	7,677	7,514
Manager limit 150,000	69	1.000	0.998	69	0	0
Senior Manager limit 250,000	4,259	1.000	0.894	0	4,259	4,161
Round multiple 5,000	0	1.000	1.000	0	0	0
Round multiple 50,000	43	1.000	0.999	43	0	0
Six-day week	35	1.000	0.999	35	0	0
Strict policy	12,274	1.000	0.693	0	12,427	12,012
Lenient policy	380	1.000	0.991	380	0	0
Version 2.6	302	1.000	0.992	302	0	0

TABLE VI
MEASURED: THE APPLICATION BEFORE AND AFTER ADOPTING THE STRICT POLICY DOCUMENT (20,000 INVOICES)

	Version 2.5	Strict policy
Findings stored	300	6,546
Invoices waiting for review	240	6,353
Invoices approved automatically	19,760	13,647
Findings under rule 5.1.1 (approval limit)	30	6,060
Findings under rule 4.1.1 (same-day splitting)	30	216
Findings under rule 2.1.1 (amount match)	30	60

TABLE VII
MEASURED: THROUGHPUT OF THE CONTROL STAGE FOR 300,000 INVOICE AUDITS

Worker processes	Seconds	Invoices per second	Speed-up
1	5.29	56,660	1.00
2	3.10	96,724	1.71
4	2.50	119,780	2.12
8	1.74	172,226	3.04

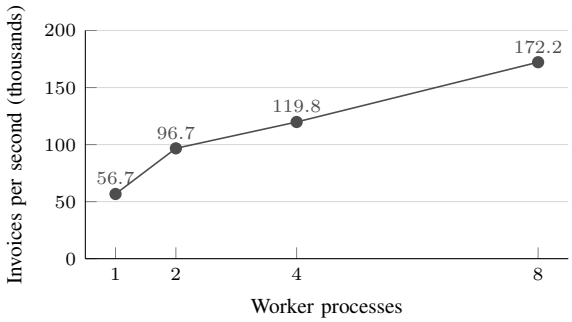


Fig. 5. Measured. Throughput of the control stage against the number of worker processes on one laptop (median of three runs).

56,660 invoices per second with one process to 172,226 with eight, a speed-up of 3.04. The speed-up is below the number of processes because the processor has two performance cores and eight slower efficiency cores.

With storage. These figures are the control stage in memory. With every result, finding and audit-trail entry written to the data store, one process audited 20,000 invoices in about one minute, which stored 60,000 trail entries.

F. Detection Against Classical Detectors (Measured)

The controls were compared with an isolation forest [5] and a one-class SVM [6], both from scikit-learn [20], on three benchmark ledgers of 5,000 invoices and on the ledger of 20,000 invoices. Each benchmark ledger holds 420 anomalous invoices (8.4%) in fourteen classes and 120 hard negatives, that is, clean invoices made to resemble an anomaly; the large ledger holds the same 420 anomalies among 20,000 invoices (2.1%). The isolation forest is fitted without labels and the one-class SVM on clean invoices. Their decision thresholds are set by five-fold cross-fitting: each fifth of a ledger is decided with the threshold that maximises F1 on the other four fifths, so the detectors use labels that the controls never see, and no invoice is decided with a threshold tuned on its own label. Because a detector names no rule, an invoice counts as detected when a system reports it at all.

Table VIII gives the result. On every ledger the controls have the highest recall and F1 and a precision of 1.000, and they raise no finding on any clean invoice, hard negatives included; on the benchmark ledgers the detectors flag 36 to 56% of the hard negatives. McNemar's exact test on the per-invoice decisions, Holm-adjusted, favours the controls over each detector on every benchmark ledger ($p < 10^{-50}$). At the realistic anomaly rate of the large ledger the isolation forest falls to an F1 of 0.211, while the controls keep 0.833: their decisions do not depend on how rare anomalies are.

Table IX shows where the difference comes from. The controls report every invoice of the ten rule-defined classes, which is expected, since they implement the definitions by which these anomalies were injected; this part is a check

of the implementation, not a discovery. The detectors find some classes well and others hardly at all: the one-class SVM misses almost every invoice with a malformed or mismatched GSTIN or a missing reference (R4, R5, R8), and both miss most invoices from suspended vendors (R9). Of the judgement classes J2 to J4, which need reading, the detectors report at most 22% and the controls none, by design. Both detectors report every near-duplicate (J1), whose copied amount and vendor make it statistically unusual; the controls leave J1 to the agents.

G. Policies in Their Own Wording (Measured)

Pattern extraction refuses a policy that is not in the wording of the template. For such a document the platform offers a model-assisted path with three safeguards. A model proposes each rule's passage and each threshold, together with the words of the document it was taken from; code then checks that every quoted passage occurs in the document word for word and that every value is written in the quote of its rule (as digits, in Indian digit grouping, or as a number word); a value that fails is held back, and a named person confirms the rest and supplies what is missing before anything is adopted.

We wrote six policy documents in two styles that share no wording and no rule numbers with the template: three memoranda in prose and three handbooks with lettered clauses, number words and Indian digit grouping, each stating the reference, the strict or the lenient policy. A local model (Qwen2.5, 7B parameters, 12,288-token context, temperature 0) read each document in one call. Table X gives the outcome against the policy each document was written from. The model proposed 80 of 84 values correctly and located 92 of 96 rules. The check accepted 56 right values and held back 24 right ones, all of them approval limits whose quote either did not contain the amount or was not found in the document; the reviewer supplies these. It caught two wrong values and let two pass: in both handbooks the model added the status "Blacklisted" to the blocked registration statuses, and the word occurs in the quoted passage, so only the confirming person can stop it. After confirmation, four of the six documents yield exactly the intended policy. The six documents are short and written by the authors of the system; this measures one model on them, not policies in general.

H. Pilot of the LLM Stages (Measured on 20 Invoices)

The LLM stages of the base system were run with the same local model on 20 invoices of a development ledger that is not used for any reported benchmark: one invoice of each of the fourteen anomaly classes, one of each of five hard-negative types, and one ordinary invoice. The configurations are C1, the controls alone; C2, a single prompt over the whole policy with no controls; C3, the controls and the three specialist agents; C4, C3 with the challenger; and C5, C4 with a confidence term that counts how many of three re-runs at temperature 0.7 repeat a finding and sends findings of low confidence to sampled review instead of escalation. Table XI gives the result. *Twenty invoices are a pilot*: the intervals are wide, and no difference between configurations is significant (McNemar's exact test, Holm-adjusted; C3 against C2, $p = 0.196$). The pilot shows that every stage runs against a real model and what an invoice costs; it does not estimate how well a model audits.

The single prompt. C2, which reads the whole policy and has no controls, flagged 18 of the 20 invoices, raised five wrong findings on the ordinary invoice and wrong rule-defined findings on most others (macro-F1 0.095 on the rule-defined classes), and two of its stages failed. Only 5 of its 75 findings named a rule the invoice violates.

The agents. With the controls first (C3), every rule-defined class was decided right, and the agents could only add findings under the four judgement rules. They found the goods-receipt remark (J4, with the remark as evidence) and the near-duplicate (J1), missed the structuring across days (J2), and cited the wrong rule for the description mismatch (J3). They also raised a wrong near-duplicate finding on three of the five hard negatives. Of the findings the agents raised, 0.111 named a rule the invoice violates.

The challenger (C4) removed nine agent findings, most of them extra findings on invoices that the controls had already flagged, and one of them the right J1 finding; it kept the wrong near-duplicate findings on the three hard negatives.

Confidence (C5) sent two invoices to sampled review instead of escalation, both of them true anomalies, among them the J3 invoice whose wrong-rule finding was repeated by only one of three re-runs. The wrong findings on the hard negatives had confidence 0.67, 0.67 and 1.00 and stayed escalated. The confidence term measures how stable a finding is, not whether it is right.

Cost. The agents need three model calls per invoice, plus the challenger when an agent raises a finding (3.60 per invoice under C4), and C5 adds the re-runs (4.95). On the laptop's CPU one call took about five minutes, so the benchmark, about 1,200 invoices per ledger, needs a GPU or a hosted endpoint.

Copilot. Through the PostgreSQL store, the copilot was asked why an escalated invoice of the large ledger was flagged. Its first answer named the wrong rule: retrieval by the wording of the question had ranked the duplicate rules first, and the model followed the passages instead of the audit record it was also given. After the passages of the rules cited in the audit record were placed first and the record was declared authoritative, it named the right rule, with the goods-receipt remark and the page of the policy.

I. Expected Results for What Is Not Yet Measured

The LLM stages of the base system have been run only in the pilot of Section VIII-H, which is too small to measure them. Table XII states what we expect on the benchmark ledgers, with the basis of each expectation and the observation that would refute it; the last column shows what the pilot observed, for orientation only. *The expected values are fixed before measurement. They are not results.*

The pilot cannot confirm or refute X2 and X3, but it shows that the 7B model used for development is far from both: on the benchmark, the agents need a larger model, and until

TABLE VIII
MEASURED: INVOICE-LEVEL DETECTION. BENCHMARK: MEAN OF THREE LEDGERS (LOWEST AND HIGHEST LEDGER IN PARENTHESES). LARGE LEDGER: VALUE [95% BOOTSTRAP INTERVAL]

	Benchmark ledgers: 3 × 5,000 invoices, 8.4% anomalous			Large ledger: 20,000 invoices, 2.1% anomalous		
Measure	Isolation forest	One-class SVM	Controls	Isolation forest	One-class SVM	Controls
Recall	0.577 (0.562–0.598)	0.464 (0.431–0.502)	**0.714**	0.162 [0.129, 0.200]	0.402 [0.359, 0.448]	**0.714** [0.672, 0.757]
Precision	0.501 (0.470–0.529)	0.543 (0.499–0.588)	**1.000**	0.301 [0.248, 0.362]	0.460 [0.410, 0.510]	**1.000** [1.000, 1.000]
F1	0.536 (0.526–0.549)	0.498 (0.497–0.501)	**0.833**	0.211 [0.170, 0.253]	0.429 [0.389, 0.470]	**0.833** [0.804, 0.862]
False positives, ordinary	0.044 (0.038–0.052)	0.025 (0.017–0.033)	**0.000**	0.007 [0.006, 0.009]	0.005 [0.004, 0.006]	**0.000** [0.000, 0.000]
False positives, hard negatives	0.403 (0.358–0.442)	0.481 (0.417–0.558)	**0.000**	0.054 [0.026, 0.085]	0.412 [0.353, 0.481]	**0.000** [0.000, 0.000]
Share of invoices flagged	0.097	0.072	0.060	0.011	0.018	0.015

TABLE IX
MEASURED: SHARE OF EACH ANOMALY CLASS REPORTED (MEAN OF THE THREE BENCHMARK LEDGERS, 30 INVOICES PER CLASS AND LEDGER)

	Rule-defined classes (decided by the controls)										Judgement (agents)			
System	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	J1	J2	J3	J4
Isolation forest	0.37	0.62	1.00	0.96	0.93	0.89	0.40	0.77	0.20	0.68	1.00	0.22	0.02	0.02
One-class SVM	1.00	0.83	0.98	0.03	0.03	0.98	0.84	0.03	0.10	0.59	1.00	0.07	0.01	0.00
Controls	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	0.00	0.00	0.00	0.00

R1 amount, R2 quantity, R3 exact duplicate, R4 GSTIN format or state code, R5 GSTIN against the vendor master, R6 same-day splitting, R7 approval limit, R8 missing reference, R9 suspended vendor, R10 weekend or round amount; J1 near-duplicate, J2 structuring across days, J3 description against the PO, J4 goods-receipt remark.

TABLE X
MEASURED: THRESHOLDS PROPOSED BY A LOCAL 7B MODEL FROM SIX POLICIES IN THEIR OWN WORDING (14 VALUES AND 16 RULES PER DOCUMENT)

	Memoranda	Handbooks	All
Values	42	42	84
Right, verified by the code check	28	28	56
Right, held back for the reviewer	12	12	24
Wrong, caught by the check	2	0	2
Wrong, passed the check	0	2	2
Rules located	47/48	45/48	92/96
Seconds per document	297–393	282–310	

TABLE XI
MEASURED PILOT: 20 INVOICES, LOCAL 7B MODEL ON A LAPTOP CPU. NOT A BENCHMARK RESULT

	C1	C2	C3	C4	C5
Invoice-level recall	0.714	0.929	0.929	0.857	0.857
Invoice-level precision	1.000	0.722	0.812	0.800	0.800
Macro-F1, rule-defined classes	1.000	0.095	1.000	1.000	1.000
Judgement classes found (of 4)	0	1	2	1	1
Hard negatives flagged (of 5)	0	4	3	3	3
Ordinary invoice flagged (of 1)	0	1	0	0	0
Model findings naming a violated rule	–	0.067	0.111	0.111	0.111
Invoices sent to sampled review	0	0	0	0	2
Model calls per invoice	0	1.15	3.00	3.60	4.95
Tokens per invoice	0	5,835	9,557	12,005	16,779

one meets X2 and X3 the judgement rules stay with human reviewers. The pilot agrees with X4.

J. Discussion

The adherence of 1.000 must be read for what it is. The controls and the ground truth are two implementations of the same written rules by the same authors. That they agree on every invoice under every policy shows that each document was read correctly and that no control keeps a threshold of its own. It verifies the mechanism; it is not a detection result, and it says nothing about whether the rules are the right ones.

The quantities that are not fixed by construction are those of the frozen system. They show that ordinary revisions of a procurement policy move between a handful and almost a third of the decisions on these ledgers, and that a tightening turns into missed escalations. A system that must be edited by hand to follow its policy carries this risk for as long as the edit is late or wrong.

The comparison with the classical detectors (Section VIII-F) is not a contest between methods of one kind. A detector learns what is usual and knows no policy; the controls know the policy and nothing else. Where the policy defines the exception, the controls decide it exactly and explain it with a rule and its evidence, while a detector misses violations that look ordinary and flags legitimate invoices that look unusual, such as the hard negatives. Detectors remain useful for what no rule describes; the near-duplicates, which both detectors report and the controls leave to the agents, are an example.

TABLE XII
EXPECTED VALUES (NOT MEASURED), FIXED BEFORE MEASUREMENT, WITH THE OBSERVATION OF THE 20-INVOICE PILOT FOR ORIENTATION

ID	Quantity	Expected value	Basis	Refuted if	Pilot, 20 invoices
X1	Policy adherence of the controls under a further policy variant in the same template	1.000	Measured 1.000 on all 14 variants; the mechanism is deterministic	Any invoice is decided against the policy in force	not applicable
X2	Recall of the LLM agents on the four judgement rules	at least 0.70 (target)	AuditBench: LLMs identify errors when given the transaction data [2]; each agent sees only the records of its rule	Below 0.70 with the chosen model; the rule then stays with human reviewers	2 of 4 classes (C3)
X3	Agent findings that name a rule the invoice violates	at least 0.90 (target)	An agent may cite only the rules of its remit, and the rule text is supplied; AuditBench reports weak citation without such limits [2]	Below 0.90	0.111 (C3 to C5)
X4	LLM calls per invoice in the base system	3, plus 1 when an agent raises a finding	One call per specialist agent; the challenger runs only when there is a finding to test	More calls are needed for valid output	3.00 (C3), 3.60 (C4)

The pilot (Section VIII-H) shows why the controls come first. Without them (C2), the model read the whole policy and raised wrong rule-defined findings on almost every invoice. With them, the rule-defined classes are decided exactly, and the model's errors are confined to the four judgement rules, where its findings go to a person. The challenger removed most unsupported findings but also a right one, and the confidence term demoted a finding that was rarely repeated, not one that was wrong. Neither replaces the reviewer.

With respect to the base papers: AuditBench [2] finds that LLMs cite the governing rule poorly, and in our pilot only 0.111 of the agents' findings named a rule that the invoice violates. For the twelve rules handled by controls the citation is no longer produced by a model at all; it is attached by the control that implements the rule. The regulatory traceability that the survey of Liu et al. [3] names as a challenge is met for these rules by construction: each finding leads to a rule, a document, a page and the field values.

IX. LIMITATIONS

Template wording. Pattern extraction reads only the wording of one template and refuses other documents with reasons. The model-assisted path for policies in their own wording was measured on six short documents written by the authors, with one 7B model; two wrong values passed its code check, so the confirming person remains a required safeguard, not a formality.

Coverage. The system executes only rules for which it has a control. A new threshold needs only a new document; a rule with new semantics needs a new control.

Synthetic data. The ledgers are synthetic. The counts describe how these ledgers react to these changes, not the books of a particular organisation.

One machine. All services and all worker processes ran on one laptop. The platform is built from separable services, but its behaviour on several machines, and the throughput of the LLM stages, have not been measured.

Judgement rules. The experiments cover the twelve rule-defined clauses. The four judgement rules are assigned to LLM agents that have been run only in a pilot of 20 invoices, with a 7B model on a laptop CPU; their accuracy on the benchmark has not been measured.

Circularity. The controls and the ground truth implement the same written rules, so the perfect scores of the controls on the rule-defined classes verify the implementation; they are not evidence that the rules catch real fraud.

Access control. The web application has one shared token and no user accounts; the reviewer's name on a decision is typed, not authenticated.

Base papers. The two base papers are described from their published abstracts.

X. CONCLUSION AND FUTURE WORK

We presented a multi-agent platform for forensic audit and compliance in procurement, built from separate services around a shared data store. Its base is a multi-agent audit pipeline that follows AuditBench and the survey of LLM-based multi-agent systems in finance, to which we added one feature: deterministic controls that are configured by the organisation's policy document. The feature is implemented and measured. From fourteen policy documents the system read every threshold correctly and decided 40,000 invoices under each policy without a wrong decision, while a system with fixed thresholds decided up to 30.7% of them against the policy in force. A change of policy takes effect in seconds and requires no change to code or prompts. On three benchmark ledgers the controls reached an F1 of 0.833 at a precision of 1.000 and raised no finding on a clean invoice, where an isolation forest and a one-class SVM reached 0.536 and 0.498 and flagged 36 to 56% of the hard negatives. The control stage costs about 18 microseconds per invoice whatever the size of the ledger, and eight worker processes on one laptop audited 172,000 invoices per second. For policies in their own wording, a local 7B model proposed 80 of 84 thresholds correctly, and a check by code with a person's confirmation decides what is adopted. A pilot of 20 invoices ran every LLM stage end to end and showed that a single prompt without controls errs on almost every invoice, while with the controls

first the model's errors stay within the judgement rules, where a person reviews them. Reviewers use the platform through a REST API and a web application with three-dimensional views of vendor risk, of the pipeline and of the effect of a policy change.

Future work is the measurement of the LLM stages on the benchmark ledgers with a larger model, against the expectations of Table XII; a deployment of the services on several machines; user accounts and roles in the web application; and an evaluation on real, anonymised procurement data.

Data and Code Availability

The platform, the synthetic ledgers, the fourteen policy documents and the scripts that produce every measured number of this paper are available at https://github.com/Chandana909/major.project.

Acknowledgment

The authors thank the Department of Information Technology, Chaitanya Bharathi Institute of Technology, Hyderabad, for the academic environment and resources that supported this work.

References

[1] M. A. Vasarhelyi and F. B. Halper, "The continuous audit of online systems," *Auditing: J. Pract. Theory*, vol. 10, no. 1, pp. 110–125, 1991. [Online]. Available: https://doi.org/10.2308/ajpt-9703192366

[2] R. Wang, J. Liu, W. Zhao, S. Li, and D. Zhang, "AuditBench: A benchmark for large language models in financial statement auditing," in *AI for Research and Scalable, Efficient Systems* (Communications in Computer and Information Science). Singapore: Springer, 2025, pp. 59–81. [Online]. Available: https://doi.org/10.1007/978-981-96-8912-5_3

[3] Q. Liu, L. Yao, Z. Wang, Y. Yang, Y. Tang, D. Cheng, and C. Jiang, "Large language model-based multi-agent systems for financial markets simulation: A survey," *Sci. China Inf. Sci.*, vol. 69, no. 7, Art. no. 171201, 2026. [Online]. Available: https://doi.org/10.1007/s11432-026-4986-x

[4] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, 2009. [Online]. Available: https://doi.org/10.1145/1541880.1541882

[5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, 2008, pp. 413–422. [Online]. Available: https://doi.org/10.1109/ICDM.2008.17

[6] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Comput.*, vol. 13, no. 7, pp. 1443–1471, 2001. [Online]. Available: https://doi.org/10.1162/089976601750264965

[7] L. Long, S. Xu, A. Li, L. Chen, and X. Gao, "An English financial audit anomaly detection system for construction industry based on machine learning," *Discover Artif. Intell.*, 2026. [Online]. Available: https://doi.org/10.1007/s44163-026-01664-8

[8] L. Hillebrand *et al.*, "Improving zero-shot text matching for financial auditing with large language models," in *Proc. ACM Symp. Document Eng.*, 2023, pp. 1–4. [Online]. Available: https://doi.org/10.1145/3573128.3609344

[9] A. Berger *et al.*, "Towards automated regulatory compliance verification in financial auditing with large language models," in *Proc. IEEE Int. Conf. Big Data*, 2023, pp. 4626–4635. [Online]. Available: https://doi.org/10.1109/BigData59044.2023.10386518

[10] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023. [Online]. Available: https://doi.org/10.1145/3571730

[11] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," in *Proc. 33rd Int. Joint Conf. Artif. Intell. (IJCAI)*, 2024, pp. 8048–8057. [Online]. Available: https://doi.org/10.24963/ijcai.2024/890

[12] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, PMLR vol. 235, 2024. [Online]. Available: https://proceedings.mlr.press/v235/du24e.html

[13] E. Mosqueira-Rey, E. Hernández-Pereira, D. Alonso-Ríos, J. Bobes-Bascarán, and Á. Fernández-Leal, "Human-in-the-loop machine learning: A state of the art," *Artif. Intell. Rev.*, vol. 56, no. 4, pp. 3005–3054, 2023. [Online]. Available: https://doi.org/10.1007/s10462-022-10246-w

[14] Artifex Software, "PyMuPDF," software. [Online]. Available: https://github.com/pymupdf/PyMuPDF

[15] ReportLab, "ReportLab open-source PDF toolkit," software. [Online]. Available: https://pypi.org/project/reportlab/

[16] LangChain, "LangGraph," software. [Online]. Available: https://github.com/langchain-ai/langgraph

[17] S. Ramírez, "FastAPI," software. [Online]. Available: https://github.com/fastapi/fastapi

[18] Poimandres, "React Three Fiber: A React renderer for three.js," software. [Online]. Available: https://github.com/pmndrs/react-three-fiber

[19] R. Cabello *et al.*, "three.js: JavaScript 3D library," software. [Online]. Available: https://github.com/mrdoob/three.js

[20] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html